Vom Maurer zum Architekten: Programmieren im KI-Zeitalter

Hartmann, W. und Näf, M.
DOI: 10.18420/ibis-04-02-04

Zusammenfassung

Künstliche Intelligenz verändert den Programmierunterricht grundlegend. Viele Informatiklehrpersonen fragen sich, welche Rolle das eigenhändige Schreiben von Code künftig noch hat, welche Kompetenzen zentral bleiben und wie sich vermeiden lässt, dass Lernende einfach KI-generierten Code übernehmen, ohne ihn zu verstehen, sodass der Lerneffekt gleich null bleibt. Der Artikel diskutiert diese Fragen und zeigt, wie sich der Fokus des Programmierens von der reinen Codeproduktion hin zu Spezifikation, Systemverständnis, kritischer Beurteilung und algorithmischem Denken verschiebt.

Programmieren als Teil der Allgemeinbildung

Wer digitale Technologien nicht nur nutzen, sondern verstehen und kritisch reflektieren will, muss sich mit Algorithmen und Programmen auseinandersetzen. Aus diesem Grund ist das Programmieren ein Pfeiler des Informatikunterrichts. Programmieren in der Schule verfolgt dabei andere Ziele als die Ausbildung von Softwareentwicklerinnen: Es geht nicht um komplexe Informatiksysteme, sondern um das Verständnis informatischer Denkweisen anhand von Programmen im überschaubaren Rahmen.

Programmieren lässt sich mit den Bildungszielen anderer allgemeinbildender Fächer vergleichen. Man muss nach der Schule nicht programmieren können – genauso wenig, wie man zum Beispiel keine mathematischen Beweise führen können muss. Aber man sollte programmiert und den einen oder anderen Beweis geführt haben. Wer selbst einfache Programme entwickelt, Schleifen formuliert, Bedingungen definiert oder Fehler sucht, gewinnt Einsichten in die Funktionsweise digitaler Systeme, die durch die reine Nutzung digitaler Werkzeuge nicht entstehen können. Der Kern des Programmierens im Informatikunterricht liegt nicht im Auswendiglernen von Syntax oder Befehlen einer bestimmten Programmiersprache. Wichtig ist die Fähigkeit, Probleme zu analysieren, Lösungen zu entwickeln, Abläufe zu strukturieren und komplexe Aufgaben so in überschaubare Teilschritte zu zerlegen, dass sie von einem Computer ausgeführt werden können.

Die Geschichte des Programmierens kann dabei als Entwicklung zunehmender Abstraktion verstanden werden, die mit den neuerdings verfügbaren KI-gestützten Entwicklungswerkzeugen eine neue Stufe erreicht. Bildlich gesprochen wandelt sich dabei die Rolle der Programmierenden vom Maurer, der Stein auf Stein setzt, zum Architekten, der entwirft, beauftragt und prüft.

Nachfolgend beschreiben wir kurz die verschiedenen Entwicklungsstufen unter dem Gesichtspunkt der genutzten Technologien und der geforderten Kompetenzen – und gehen der Frage nach, welche Kompetenzen Lernende im KI-Zeitalter noch selbst beherrschen müssen.

Maschinennahe Programmierung

Die Anfangszeiten der Informatik waren geprägt durch die maschinennahe Programmierung. Der Computer wurde direkt auf technischer Ebene gesteuert. Programmierer arbeiteten mit Maschinensprache oder Assembler und mussten sich mit Registern, Speicheradressen und der internen Funktionsweise des Prozessors auseinandersetzen. Um Programme entwickeln zu können, war es notwendig zu verstehen, wie Daten gespeichert werden und wie der Prozessor Befehle Schritt für Schritt abarbeitet. Diese Form des Programmierens verlangte die vollständige Kontrolle über jedes Detail – ohne Sicherheitsnetz: Es gab weder Compiler-Prüfungen noch Fehlermeldungen; kleine Fehler äusserten sich als beinahe unerklärliches Fehlverhalten.

Höhere Programmiersprachen

Programmiersprachen wie BASIC, C, Java oder Python ermöglichten es, Algorithmen mit abstrakteren Befehlen zu formulieren. Die Übersetzung in Maschinencode wurde vom Compiler oder Interpreter übernommen. Dadurch wurde Programmieren für deutlich grössere Anwend-

ergruppen zugänglich. An die Stelle des Hardware-Wissens trat die Beherrschung von Syntax, Datenstrukturen und Debugging; algorithmisches Denken blieb zentral. Obwohl viele technische Details der Hardware verborgen blieben, lag die eigentliche Problemlösung weiterhin beim Menschen. Der Maurer erhielt besseres Werkzeug.

Entwicklungsumgebungen, Bibliotheken und Frameworks

Mit modernen Entwicklungsumgebungen nahm die Abstraktion erneut deutlich zu. Der Schwerpunkt verlagerte sich von der vollständigen Eigenentwicklung einzelner Funktionen hin zur Wiederverwendung vorhandener Komponenten und Werkzeuge. Dazu gehörten beispielsweise das Entwerfen grafischer Benutzeroberflächen, Datenbankzugriffe oder Netzwerkkommunikation. Programmierer mussten diese Funktionen nicht mehr selbst implementieren, sondern konnten auf bestehende Lösungen zurückgreifen. Neben der Problemanalyse wurden insbesondere Systemdenken, Architekturverständnis sowie die Fähigkeit wichtig, Bibliotheken und Schnittstellen gezielt zu nutzen. Dazu kam die Beurteilung: Entwicklerinnen mussten entscheiden, ob sie eine Funktion selbst entwickeln oder auf eine bestehende Lösung setzen – und deren Qualität und Eignung einschätzen können. Statt jeden Stein selbst zu setzen, baute man zunehmend mit Fertigteilen.

KI-gestütztes Programmieren

Mit KI-gestützten Entwicklungswerkzeugen entsteht gegenwärtig eine weitere Abstraktionsstufe. Anforderungen müssen nicht mehr vom Menschen in eine Programmiersprache übersetzt werden: Sie werden in natürlicher Sprache beschrieben, die Übersetzung in Code übernimmt die KI. Coding-Assistenten und autonome Agenten – etwa GitHub Copilot, OpenAI Codex, Claude Code oder Mistral Code – übernehmen dabei die eigentliche Codeproduktion. Die Rolle des Menschen verändert sich dadurch grundlegend: Entwicklerinnen formulieren Anforderungen, die KI generiert darauf aufbauend Programmcode, Testfälle, Benutzeroberflächen, Architekturvorschläge oder Dokumentationen. Nun übernimmt gleichsam eine ganze Baufirma die Ausführung.

Teilweise wird bei dieser Methode der Softwareentwicklung das Programmieren bereits heute weitgehend als Black Box behandelt. Der Fokus liegt in diesem Fall ausschliesslich auf der präzisen Spezifikation der gewünschten Lösung sowie auf umfangreichen automatisierten Tests. Solange alle Tests erfolgreich durchlaufen werden, spielt der konkret erzeugte Programmcode nur noch eine untergeordnete Rolle und wird teilweise kaum mehr gelesen. Konsequent zu Ende gedacht bedeutet dieses Vorgehen eine weitere Abstraktionsstufe. Karpathy und andere gehen noch einen Schritt weiter und lassen gar kein Programm mehr erzeugen: Die KI führt die beschriebene Aufgabe direkt aus – die Beschreibung selbst ist das Programm, das KI-System seine Laufzeitumgebung (Karpathy 2025). Ob sich dieser Ansatz durchsetzt, ist offen: Erzeugter Programmcode bleibt vorerst schneller, günstiger und vor allem besser überprüfbar.

Eine gute und humorvolle Übersicht zum Thema findet sich im Beitrag «Artificial Intelligence for Software Engineering: From Probable to Provable» von Bertrand Meyer (Meyer 2026). Er betont, dass die Softwareentwicklung der Zukunft die Kreativität der KI mit der Striktheit formaler Spezifikation und Verifikation kombinieren muss.

Was heisst das für den Programmierunterricht?

Trotz aller Abstraktion bleibt der Kerngedanke beim Programmieren über alle Entwicklungsstufen hinweg identisch: Ein Problem wird analysiert, strukturiert und in eine Form überführt, die ein Computer ausführen kann. Doch die Ebene der Abstraktion und damit die Schwerpunkte der geforderten Kompetenzen ändern sich. Dabei zeigt sich ein wiederkehrendes Muster: Jede Stufe verbirgt einen Teil der bisherigen Arbeit – die Maschinensprache die Elektronik, die Hochsprachen die Hardware, die Bibliotheken die Implementierung von Standardfunktionen, die KI nun die Codeproduktion selbst. Was verborgen wird, verschwindet jedoch nicht: Die Verantwortung dafür wandert nach oben, vom Selbermachen zum Beurteilen. Nur eines wandert nie: Die Frage, ob das richtige Problem gelöst wird, bleibt auf jeder Stufe beim Menschen. Und keine Abstraktion ist ganz dicht: Wo sie versagt – eine Bibliothek verhält sich unerwartet, generierter Code tut das Falsche –, muss man eine Ebene tiefer steigen können. Eigene Programmerfahrung bleibt deshalb weiterhin wichtig. Wer nie selbst programmiert hat, wird auch Mühe haben, Programmcode zu lesen, zu verstehen und kritisch zu beurteilen. Die Fähigkeit zur Modellbildung sowie algorithmisches Denken, also Computational Thinking, rücken aber in den Vordergrund.

Tabelle 1: Abstraktionsstufen des Programmierens – was verborgen wird und was bleibt

Stufe	Was verborgen wird	Was beim Menschen bleibt
Maschinennahe Programmierung	die Elektronik: Strom, Transistoren, Schaltkreise	jedes Detail: Speicher, Register, Abläufe
Höhere Programmiersprachen	die Hardware	Algorithmen, Syntax, Datenstrukturen
Bibliotheken und Frameworks	die Implementierung von Standardfunktionen	Auswahl, Komposition, Architektur
KI-gestütztes Programmieren	die Codeproduktion	Spezifikation, Lesen, Beurteilen
(absehbar: agentische Black Box)	der Code selbst – bis hin zur direkten Ausführung ohne erzeugtes Programm	Spezifikation und Verifikation

Eine ähnliche Entwicklung erfuhr die Mathematik. Mit der Verbreitung von Taschenrechnern verloren schriftliches Addieren, Multiplizieren oder Wurzelziehen an Bedeutung. Dennoch bleibt ein Verständnis der Rechenoperationen sowie ein gewisses Zahlengefühl unverzichtbar, um Resultate beurteilen und Fehler erkennen zu können. Auf höheren Stufen werden längst Computeralgebra-Systeme genutzt. Der Mathematikunterricht war gefordert, dieser Entwicklung laufend Rechnung zu tragen. Analog steht der Informatikunterricht vor der Frage, was Lernende beim Programmieren weiterhin von Hand beherrschen müssen. In einem Punkt hinkt der Vergleich allerdings: Taschenrechner und Computeralgebra-Systeme rechnen deterministisch und zuverlässig richtig, KI-Systeme dagegen arbeiten probabilistisch und liefern keine Garantie für korrekte Resultate. Die Fähigkeit, Ergebnisse kritisch zu beurteilen, wiegt beim Programmieren mit KI deshalb noch schwerer als das Zahlengefühl in der Mathematik.

Natürliche Sprache als algorithmische Beschreibungsebene

«Erst lesen, dann schreiben» ist eine anerkannte Unterrichtstechnik. Analog zum Lesen- und Schreibenlernen in der Primarschule sollen die Schülerinnen zunächst lernen, bestehenden Programmcode zu lesen, zu verstehen und zu interpretieren, bevor sie selbst Programme schreiben. Durch das Lesen solcher Programme erkennen die Lernenden typische Programmstrukturen, Abläufe, Algorithmen und Lösungsstrategien. Der Ansatz betont damit, dass Programmierverständnis nicht primär durch das Produzieren von Code entsteht, sondern durch das Verstehen bestehender Lösungen und deren schrittweise Analyse. Für den Unterricht etabliert hat sich das PRIMM-Modell (Sentance and Waite 2017), das Lernende dazu anleitet, zunächst bestehenden Code vorherzusagen (Predict), auszuführen (Run), zu untersuchen (Investigate) und zu modifizieren (Modify), bevor sie eigenen Code schreiben (Make). Im KI-Zeitalter wird das Analysieren und Lesen von Programmcode wichtiger als das Schreiben. Strukturierte natürliche Sprache übernimmt die Rolle des Pseudocodes – mit dem Unterschied, dass sie direkt von der KI in Code übersetzt wird. Damit vollzieht sich ein Übergang von streng formalen hin zu semi-formalen Repräsentationen, die sowohl für Menschen verständlich als auch für KI-Systeme interpretierbar sind. Die Rolle der Programmierenden verändert sich vom reinen «Code-Schreiber» bzw. «Maurer» hin zum Architekten und Gestalter von Lösungsprozessen. Gute Architektinnen brauchen aber selbstverständlich weiterhin Verständnis für Materialien und Konstruktion. Und anders als eine eingespielte Baufirma arbeitet die KI zwar schnell, aber nicht zuverlässig. Wer mit ihr baut, ist deshalb nicht nur Architekt, sondern auch Bauleiter: Zur neuen Rolle gehört die Bauabnahme – das kritische Prüfen des ausgeführten Werks.

Programmieren in Lehrplänen und Lehrmitteln

Die neue Ausrichtung des Programmierunterrichts bedingt Anpassungen der heutigen Lehrpläne und Lehrmittel. Nachfolgend wird ein denkbarer, modularer Lehrplan für rund 100 Lektionen im Fach Informatik auf Stufe Gymnasium oder Berufsschule (Klasse 10–13) skizziert.

Im ersten Modul «Vom Gedanken zur Spezifikation» lernen die Schüler, vage Ideen in strukturierte Beschreibungen zu übersetzen und sammeln erste Erfahrungen mit Coding-Assistenten. Präzise Beschreibungen führen dabei fast zwangsläufig zu den Grundbausteinen des Programmierens: Wer ein Spiel spezifiziert, formuliert Bedingungen («wenn der Spieler gewinnt, dann …»), Wiederholungen («für jede Runde …») und Fallunterscheidungen – zunächst in natürlicher Sprache.

Das zweite Modul «Werkzeuge und ihre Grenzen» nutzt aktuelle KI-Entwicklungswerkzeuge als Anschauungsmaterial für dauerhafte Konzepte: Die Lernenden erfahren, dass dieselbe

Spezifikation unterschiedliche Programme ergeben kann (Nichtdeterminismus), analysieren typische Fehlerbilder – von erfundenen Funktionen bis zu plausibel aussehendem, aber falschem Code – und lernen, Spezifikationen iterativ zu verbessern und Resultate systematisch zu überprüfen.

Im dritten Modul «Code-Reading und Qualitätssicherung» lernen die Schülerinnen, fremden Code – ob von Menschen oder von KI geschrieben – zu lesen, zu verstehen und kritisch zu beurteilen. Damit das Beurteilen nicht ohne eigene Schreiberfahrung bleibt, verändern die Lernenden KI-generierten Code von Hand und schreiben immer wieder auch kleine Programme selbst. Ergänzt wird dies durch Testing, Debugging sowie Fragen zu Sicherheit und Datenschutz beim Einsatz generativer KI-Systeme.

Im vierten Modul «Systemarchitektur und Datenmodellierung» beschäftigen sich die Lernenden mit modularen Systemen, APIs, Frontend-Backend-Kommunikation sowie Konzepten der Datenstrukturierung und Datenbanken. Den Abschluss bildet ein umfangreiches Projekt, in dem die Schülerinnen und Schüler in Teams eine eigene Anwendung von der Idee bis zur Präsentation entwickeln. Dabei sollen sie ihre Spezifikationen, Architekturentscheidungen und die Qualitätssicherung der KI-generierten Lösungen begründen und reflektieren.

Ein solcher Programmierunterricht fördert drei Kompetenzen. Erstens lernen Schülerinnen und Schüler kritisches Denken im Umgang mit KI-Systemen. KI wird dabei nicht als unfehlbare Autorität verstanden, sondern als leistungsfähiger Assistent, dessen Resultate überprüft, hinterfragt und validiert werden müssen. Zweitens gewinnt systemisches Denken an Bedeutung. Da die Hürden der Syntax weitgehend wegfallen, können Lernende sich stärker mit Software-Architektur, Modularisierung und komplexen Zusammenhängen beschäftigen. Drittens entstehen bereits nach kurzer Zeit funktionierende und vergleichsweise komplexe Anwendungen, was Motivation, Kreativität und Gestaltungsfreude der Lernenden erheblich fördern kann.

Als Einstieg in Modul 1 könnten die Lernenden zum Beispiel die Aufgabe erhalten, mit einem einzigen kurzen Prompt ein einfaches Schere-Stein-Papier-Spiel (oder ein anderes selbstgewähltes Spiel) durch einen KI-Agenten erzeugen zu lassen. Obwohl die generierten Programme meistens funktionieren – einfache Standardspiele sind in den Trainingsdaten der KI-Modelle reichlich vertreten –, zeigen sich oft typische Probleme wie fehlerhafte Logik, fehlende Funktionen oder unklare Benutzerführung. Deutlicher wird es mit einem gänzlich selbst erfundenen Spiel oder mit einer neuen Regelvariante, etwa einer vierten Geste: Was in den Trainingsdaten keine Vorlage hat, muss präzise spezifiziert werden. Die Erkenntnis lautet: KI-gestütztes Programmieren bedeutet nicht «weniger denken», sondern «präziser denken». Anschliessend kann die Lehrperson ein einfaches Raster für strukturierte Spezifikationen einführen. Dazu gehören Rolle und Kontext der KI, funktionale Anforderungen, Gestaltungsvorgaben sowie technische Einschränkungen. Die Lernenden überarbeiten daraufhin ihre ursprünglichen Prompts, ergänzen zusätzliche Anforderungen und verbessern die Spezifikationen schrittweise. Dieser Einstieg ins Programmieren verdeutlicht exemplarisch, dass im KI-Zeitalter weniger das Schreiben einzelner Syntaxelemente als vielmehr das präzise Beschreiben, Strukturieren und Überprüfen von Lösungen bedeutsam wird.

Wie wird geprüft?

Der aufgezeigte neue Zugang zum Programmierunterricht entschärft das Problem, dass Lernende Programmcode von KI-Systemen erzeugen lassen, ohne sich vertieft mit den zugrunde liegenden Lösungen auseinanderzusetzen. Wenn nicht mehr primär das eigenhändige Schreiben von Code, sondern das Verständnis von Problemlösungen im Vordergrund steht, verändern sich automatisch die Prüfungsformen. An Bedeutung gewinnen beispielsweise Code-Reviews, Spezifikationsanalysen, Fehleranalysen KI-generierter Lösungen, Architekturbegründungen, Prompt-Reflexionen oder iterative Verbesserungsaufgaben. Auch diese Leistungen lassen sich freilich KI-generieren. Prüfungsformen, die Verständnis sichtbar machen sollen, setzen deshalb verstärkt auf Dialog und direkte Beobachtung: mündliche Verteidigungen der eigenen Entscheidungen, Erklären unter Nachfragen, betreute Arbeitsphasen sowie benotete Projektarbeiten, deren Entstehung begleitet und deren Entscheidungen im Gespräch begründet werden.

Fazit

KI verändert nicht das Ziel des Programmierens, sondern die Ebene der Abstraktion. Historisch betrachtet handelt es sich um einen weiteren Schritt in einer langen Entwicklung zunehmender Abstraktion – von Maschinensprache über höhere Programmiersprachen und Frameworks hin zu KI-gestützten Entwicklungswerkzeugen.

Ansätze des hier skizzierten Programmierunterrichts werden bereits an Hochschulen, in der Informatikberufsbildung und vereinzelt auch an Schulen erprobt. Nun sind Schulen, Informatikdidaktik und Lehrmittelverlage gefordert, entsprechende Konzepte weiterzuentwickeln, Erfahrungen zu sammeln und den Programmierunterricht an die Möglichkeiten und Anforderungen des KI-Zeitalters anzupassen.

Die Herausforderung besteht nicht darin, Programmieren trotz KI zu unterrichten, sondern jene Kompetenzen zu fördern, die beim Programmieren mit KI in den Vordergrund rücken. Oder im Bild des Titels: Gefragt sind künftig weniger Maurer als Architektinnen und Bauleiter – Menschen, die entwerfen, beauftragen und prüfen können und gleichwohl wissen, wie gemauert wird.

Quellen

Karpathy, Andrej (2025). Software Is Changing (Again) ("Software 3.0"). Y Combinator, YouTube, veröffentlicht Juni 2025. https://www.youtube.com/watch?v=LCEmiRjPEtQ

Meyer, Bertrand (2026). Artificial Intelligence for Software Engineering: From Probable to Provable. Communications of the ACM, Vol. 69, No. 6, S. 46–49.

Sentance, Sue and Waite, Jane (2017). PRIMM: Exploring pedagogical approaches for teaching text-based programming in school. In: Proceedings of the 12th workshop on Primary and Secondary Computing Education, S. 113–114.

Lizenz

Dieser Artikel steht unter der Lizenz CC BY-NC 4.0 zur Verfügung.

Autoren

Werner Hartmann, werner.hartmann@phsz.ch
Pädagogische Hochschule Schwyz

Myke Näf, myke@uebermorgen.vc
Übermorgen Ventures